

Documentation of the Word Complexity Prediction Project for Machine Learning Course

2st Semester of 2024-2025

Stefan Octavia

octavia-elena@s.unibuc.ro

Branzea Malina-Alexandra

malina-alexandra@s.unibuc.ro

Abstract

This project focuses on training regression models to predict the complexity of words by analysing their morphological, syntactic and semantic characteristics within a sentence. To assign a numerical complexity score to each target word, we experimented with several regression algorithms. The most successful was LightGBM, with features extracted from spaCy's contextual embeddings, complemented by handcrafted linguistic indicators.

1 Introduction

The aim of this project was to build a model that predicts how complex a word is when used in a given sentence, based on both its form and its surrounding context. The dataset contained Romanian sentences with a labeled target word and an associated complexity score ranging between 0 and 1. Throughout the development process, we explored various strategies to extract word-level features and trained different regressors to maximize prediction performance. We opted for a hybrid approach combining dense semantic representations with interpretable surface-level features.

2 Approach

To address the task of predicting word complexity, we employed a hybrid feature engineering strategy combining both semantic embeddings and handcrafted linguistic features. The central component of our pipeline was the Romanian spaCy model called `ro_core_news_lg`, which provides 300-dimensional contextual word vectors trained on large Romanian corpora. Feature Extraction Process For each sentencetoken pair in the dataset, we extracted: A 300-dimensional contextual embedding from spaCy Morphological features: word length, digit presence, casing flags Binary indicators: is uppercase, is title case, is punctuation Part-of-speech flags: is adjective, noun, verb, adverb These features were combined into a single feature vector.

We applied standard scaling to normalize the handcrafted features before training.

2.1 Dataset Preparation

The labeled data was loaded from `train.csv`, while `test.csv` was used for submission. We split the training data into 80 percent training and 20 percent validation. Features were extracted using custom Python functions, and training pipelines were built using scikit-learn.

3 Tools and Libraries

We used the following tools and libraries to implement the pipeline:

pandas – for reading and processing CSV files

numpy – for efficient numerical operations

spaCy – for tokenization, POS tagging and embeddings

scikit-learn – for feature scaling, regression models, model evaluation

LightGBM – for efficient gradient boosting decision trees

All development was done in Jupyter Notebook using Python 3. The final test predictions were saved in a file named `submission.csv`

4 Model Evaluation

We evaluated four different regression models using the R-squared score on the validation set:

Ridge Regression: R-squared approximately 0.489

Support Vector Regression: R-squared approximately 0.402

Gradient Boosting Regressor: R-squared approximately 0.524

LightGBM Regressor: R-squared 0.538 (best result)

Each model was trained using a scikit-learn pipeline that included feature scaling and model fitting. The LightGBM model outperformed others in both accuracy and computational efficiency. Pre-

ditions for the test set were clipped to the interval [0, 1] and exported to submission.csv.

5 Model Training

The training process focused on fitting multiple regression models using the feature matrix obtained from combining contextual embeddings with hand-crafted linguistic features. We experimented with the following models: Ridge Regression: a regularized linear regression model with L2 penalty. It served as a simple and fast baseline. Support Vector Regression (SVR): a kernelbased method capable of capturing non-linear relationships, though it was slower in training. Gradient Boosting Regressor: an ensemble model based on decision trees trained sequentially to reduce prediction errors. LightGBM Regressor: an optimized and fast gradient boosting algorithm that uses histogrambased learning and leaf-wise tree growth. It proved to be the most accurate in our case. All models were integrated in a pipeline along with a standard feature scaler. We used Standard-Scaler to normalize the input features, especially important for models sensitive to input magnitude like Ridge and SVR. We trained each model on 80 percent of the training data and evaluated its performance on the remaining 20 percent (validation set). A simple loop was implemented to automate model training and comparison. The model with the highest Rsquared value on the validation set was selected as the final model. The LightGBM regressor achieved the best results in terms of both accuracy and training speed.

6 Conclusions and Future Work

This project demonstrated the benefits of combining deep semantic representations with simple linguistic features. The best performance was achieved using LightGBM, confirming the effectiveness of gradient boosting in structured regression tasks.

6.1 Conclusions and Future Work

- Efficient use of contextual embeddings with interpretable features
- A modular and clean pipeline architecture
- Successful application of a Romanian-specific language model

6.2 Conclusions and Future Work

- Lack of hyperparameter optimization (no grid search or Optuna)
- No sentence-level context features were added
- Some rare or out-of-vocabulary words had weak vector representations

6.3 Conclusions and Future Work

- Using transformer embeddings such as Romanian BERT
- Adding sentence-level and dependency-based features
- Automating parameter tuning with Optuna or cross-validation

This pipeline can be adapted to other languages and tasks with minimal effort, making it reusable and extensible for further research.

References

- Explosion AI. 2024. spaCy: Industrial-Strength Natural Language Processing in Python. Available at: <https://spacy.io/>.
- Microsoft. 2024. LightGBM: A Fast, Distributed, High Performance Gradient Boosting Framework. Available at: <https://lightgbm.readthedocs.io/>.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- A. Mihaila, C. Rau, and S. Dumitrescu. 2022. RoCore- News Language Model: Pretrained spaCy Pipelines for Romanian. Available via spaCy Model Registry. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.